

JavaScript Quiz

A Developer's Reference — Tricky Concepts & Outputs

Covers: Type Coercion · Equality · Arrays · Objects · Closures · Prototypes ·
Scope · Floating-Point · Operators & More

Built by Priya Bagde
[linkedin.com/in/priya-bagde](https://www.linkedin.com/in/priya-bagde)

Section 1 — Equality & Type Coercion

Q1.

Reference equality: arrays are objects — two literals are never the same reference.

```
console.log([1,2] == [1,2]);  
console.log([1,2] === [1,2]);
```

Output: false | false

== and === both compare references for objects/arrays, not contents.

Q2.

Comparing arrays by content — three common approaches:

```
// 1. Convert to string  
a.toString() === b.toString();  
  
// 2. Manual length + element check  
let isEqual = ((a, b) => !!a && !!b &&  
a.length === b.length &&  
a.every((ele, i) => b[i] === ele));  
  
// 3. JSON.stringify (handles nested)  
JSON.stringify(a) === JSON.stringify(b);
```

None of these is universal. Choose based on depth and type requirements.

Q3.

Truthy/falsy — an array (even [0]) is always truthy; the number 0 is falsy.

```
if ([0]) { console.log('true') } else { console.log('false') }  
if (0) { console.log('true') } else { console.log('false') }
```

Output: true | false

Q4.

Array concatenation with + uses string coercion.

```
console.log([1,2] + [3,4]);
```

Output: 1,23,4

Both arrays are toString()-coerced first: '1,2' + '3,4' = '1,23,4'.

Q5.

Type coercion with the unary + operator.

```
var a = "Hello";  
var b = +a;  
console.log(typeof a, typeof b, b);
```

Output: string number NaN

Unary + tries numeric conversion; 'Hello' yields NaN, but typeof NaN === 'number'.

Q6.

typeof null and typeof undefined — a classic JavaScript quirk.

```
console.log(typeof null);  
console.log(typeof undefined);
```

Output: object | undefined

typeof null === 'object' is a historical bug in JS preserved for backward compatibility.

Q7.

String/number arithmetic — + concatenates when either operand is a string.

```
console.log(4 + '3');  
console.log(4 - '3');
```

Output: 43 | 1

- always attempts numeric coercion; + is overloaded for strings.

Q8.

0.1 + 0.2 — floating-point representation explained.

```
console.log(0.1 + 0.2);
```

Output: 0.30000000000000004

Binary floating-point (IEEE 754) cannot represent 1/10 or 1/5 exactly — they are repeating fractions in base 2, just as 1/3 is repeating in base 10. The tiny leftovers accumulate and surface in the decimal display.

Fix: parseFloat((0.1 + 0.2).toFixed(2)) → 0.30

Q9.

Logical operators `||` and `&&` — return an operand, not just true/false.

```
console.log(0 || 1);  
console.log(1 || 2);  
console.log(0 && 1);  
console.log(1 && 2);
```

Output: 1 | 1 | 0 | 2

|| returns first truthy or last value; && returns first falsy or last value.

Q10.

`Math.min()` with no arguments returns `+Infinity`; `Math.max()` returns `-Infinity`.

```
console.log(Math.min() < Math.max());  
console.log(Math.min() > Math.max());
```

Output: false | true

Identity element: `Math.min()` = `+Infinity` (anything beats it), `Math.max()` = `-Infinity`.

Section 2 — Objects & Arrays

Q11.

Object keys are always coerced to strings. Two distinct objects share the same key '[object Object]'.

```
const a = {};  
const b = { key: 'b' };  
const c = { key: 'c' };  
a[b] = 123;  
a[c] = 456;  
console.log(a[b]);
```

Output: 456

Both b and c coerce to '[object Object]', so a[c]=456 overwrites a[b]=123.

Q12.

Object.entries / Object.keys / Object.values — iterating an object.

```
const obj = { one:1, two:2, three:3 };  
  
Object.entries(obj); // [['one',1], ['two',2], ['three',3]]  
Object.keys(obj); // ['one', 'two', 'three']  
Object.values(obj); // [1, 2, 3]
```

Q13.

Computed property names — dynamic keys in object literals.

```
const dynamic = 'flavor';  
var item = { name: 'Biscuit', [dynamic]: 'Chocolate' };  
console.log(item);
```

Output: {name: 'Biscuit', flavor: 'Chocolate'}

Q14.

Object.freeze prevents mutation (silently in non-strict mode).

```
const USER = { Name: 'Nag', age: 25 };  
Object.freeze(USER);  
USER.age = 27;  
console.log(USER.age);
```

Output: 25

Frozen objects ignore assignments silently. Use strict mode to see a TypeError.

Q15.

const protects the binding, not the value — object properties remain mutable.

```
const USER = { Name: 'Nag', age: 25 };
USER.age = 27;
console.log(USER.age);
```

Output: 27

const prevents reassignment (USER = ...) but NOT property changes.

Q16.

Four ways to empty an array.

```
arrayList.length = 0; // truncates in place – refs updated
arrayList = []; // new array – old refs unaffected
arrayList.splice(0, arrayList.length); // in place
while (arrayList.length) { arrayList.pop(); } // iterative
```

Prefer .length = 0 or .splice(0) when other references to the same array exist.

Q17.

Array.prototype IS an array — it's the hidden prototype of every array literal.

```
console.log(Array.isArray(Array.prototype));
```

Output: true

Q18.

Iterating plain objects with for...of throws — objects are not iterable by default.

```
const myMoney = { quarters:4, dimes:10, nickels:20, pennies:100 };
for (const coin of myMoney) {
  console.log(`${coin} : ${myMoney[coin]}`);
}
```

Output: TypeError: myMoney is not iterable

Use for...in, Object.keys(), or Object.entries() instead.

Q19.

Array destructuring with skipped positions.

```
const props = [  
  { age:23, name:'abreh' },  
  { age:27, name:'Yonas' },  
  { age:25, name:'sara' }  
];  
const [, { name }] = props;  
console.log(name);
```

Output: Yonas

The leading comma skips index 0; destructuring then targets index 1.

Q20.

Shuffling an array with sort() — non-deterministic output.

```
const arr = [5, 24, 120, -23, 4, 5, -34, 98];  
const shuffle = (arr) => arr.sort(() => Math.random() - 0.5);  
console.log(shuffle(arr));
```

Output: [-23, 5, 4, 98, 5, 24, -34, 120] (varies each run)

For a statistically uniform shuffle use the Fisher-Yates algorithm instead.

Section 3 — Scope, Closures & this

Q21.

var vs let inside setTimeout — var leaks to function scope, let is block-scoped.

```
// var – loop completes before callbacks fire
for (var i = 0; i < 3; i++)
  setTimeout(() => console.log(i), 1);

// let – each iteration has its own binding
for (let i = 0; i < 3; i++)
  setTimeout(() => console.log(i), 1);
```

Output: 3 3 3 | 0 1 2

var shares one binding; by the time callbacks run i === 3. let creates a fresh binding per iteration.

Q22.

IIFE return value — typeof of the return value of an immediately invoked arrow.

```
function sayHi() {
  return (() => 0)();
}
console.log(typeof sayHi());
```

Output: number

The IIFE returns 0 immediately; typeof 0 === 'number'.

Q23.

this in method vs detached function — context is set at call-site, not definition.

```
function foo() { console.log(this.a); }

var obj = { a: 'in obj', foo: foo };
var a = 'outside obj';

function doFoo(fn) { fn(); }

doFoo(obj.foo); // 'outside obj' – fn() loses obj context
doFoo.call(obj, obj.foo); // 'outside obj' – obj.foo is passed as value,
// fn() still called without obj
```

Output: outside obj | outside obj

Passing a method as an argument strips its receiver. Use .bind(obj) to retain context.

Q24.

this.logProp vs a local variable named logProp.

```
const testThis = {  
  property: 'foo',  
  logProp: function() {  
    const logProp = 'bar';  
    console.log(this.logProp); // the method itself  
  }  
};  
testThis.logProp();
```

Output: [Function: logProp]

this.logProp refers to the method on the object, not the local variable 'bar'.

Q25.

Closure captures the variable from outer scope even after outer function returns.

```
(function (a) {  
  return (function () {  
    console.log(a);  
    a = 6;  
  })();  
})(21);
```

Output: 21

Inner function closes over parameter 'a'. It logs the original value 21 before reassigning.

Q26.

typeof a variable whose value was set to typeof of an undeclared variable.

```
let y = 1;  
y = typeof x; // x undeclared → 'undefined'  
console.log(typeof y);
```

Output: string

typeof undeclared returns the string 'undefined', so y holds a string value.

Section 4 — Prototypes & Built-ins

Q27.

Extending String.prototype — custom methods on primitives via auto-boxing.

```
String.prototype.printData = () => 'Printing data';  
const data = 'Instagram';  
data.printData();
```

Output: Printing data

Accessing a property on a string primitive temporarily wraps it in a String object, enabling prototype lookups.

Q28.

Detecting plain objects by checking constructor.name.

```
function isObject(obj) {  
  return obj != null && obj.constructor.name === 'Object';  
}  
console.log(isObject({})); // true  
console.log(isObject([])); // false  
console.log(isObject(null)); // false
```

Output: true | false | false

Array constructor name is 'Array'; null short-circuits before .constructor access.

Q29.

Primitive strings are immutable — methods return new values without modifying the original.

```
let myString = "hello world";  
myString.toUpperCase(); // return value discarded  
console.log(myString.split(" "));
```

Output: ['hello', 'world']

toUpperCase() is never assigned back, so myString remains 'hello world'.

Q30.

Array.filter with Boolean constructor as a callback — removes all falsy values.

```
const values = ['web', '', 'undefined', 'javascript', null, false];  
  
values.filter((v) => Boolean(v));  
values.filter(Boolean); // shorthand
```

Output: ['web', 'undefined', 'javascript']

Note: the string 'undefined' is truthy; null, false, and "" are falsy.

Q31.

Array.find returns the first matching element (not an array).

```
const users = [
  { id:1, name:'John' },
  { id:2, name:'Wick' },
  { id:3, name:'Kim' }
];
const user = users.find(u => u.name === 'Kim');
console.log(user);
```

Output: {id: 3, name: 'Kim'}

Q32.

Reduce to sum digits of a numeric string.

```
let numbers = "01234";
let result = numbers
.split('')
.reduce((a, b) => parseInt(a) + parseInt(b));
console.log(result);
```

Output: 10

Q33.

Array.map(Number) coerces strings — empty string and non-numeric values.

```
const stringNumbers = ['123', '4.45', 'abc', '1000', '0', ''];
console.log(stringNumbers.map(Number));
```

Output: [123, 4.45, NaN, 1000, 0, 0]

Number("") === 0; Number('abc') === NaN.

Q34.

Palindrome check using split/reverse/join.

```
let word = 'madam';
word === word.split('').reverse().join('');
```

Output: true

Section 5 — Control Flow & Miscellaneous

Q35.

continue skips the current iteration; loop still produces 0, 1, 3.

```
for (let i = 0; i < 4; i++) {  
  if (i === 2) continue;  
  console.log(i);  
}
```

Output: 0 | 1 | 3

Q36.

Labelled block — foo is a label, NOT an object. Dot-access on a label is undefined.

```
var bar = 1;  
var bar = 1, foo = {};  
foo: {  
  bar: 2;  
  baz: ++bar;  
}  
console.log(foo.baz + foo.bar + bar);
```

Output: NaN

foo: is a statement label; foo stays {}. foo.baz and foo.bar are both undefined; undefined + undefined + 2 = NaN.

Q37.

Conditional property injection using spread and short-circuit evaluation.

```
let isTruthy = true;  
  
// Imperative  
if (isTruthy) { person.alias = 'Iron Man'; }  
  
// Declarative (preferred)  
const person = {  
  name: 'Tony Stark',  
  ...(isTruthy && { alias: 'Iron Man' })  
};
```

Output: {name: 'Tony Stark', alias: 'Iron Man'}

When isTruthy is false the spread receives false, which adds no properties.

Q38.

Logging variables with labels — two patterns compared.

```
let a = 10, b = 20;

// Positional – value only, label is a separate string
console.log(a, "a"); // 10 'a'

// Object shorthand – key is the variable name
console.log({ a }, { b }); // {a: 10} {b: 20}
```